

MODULE ALGORITHMIQUE ET PROGRAMMATION C - III

ALGORITHMES DE TRI RAPIDE

©B. DERDOURI

TriRapide

Module AlgoIII- L3I-S5-U-CERGY

9/25/2022

1

PLAN DE COURS

- **Tri Rapide**
 - Principe
 - Algorithme
- **Partitionner**
 - Principe
 - Choix du pivot
 - Action
 - Exemple
 - Version 1 : algorithme partitionner
 - Version 2 : algorithme partitionner
- **Exemples Tri Rapide**
- **Complexité Tri Rapide**

©B. DERDOURI

TriRapide

Module AlgoIII- L3I-S5-U-CERGY

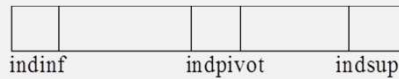
9/25/2022

2

APPROCHE DIVISER POUR MIEUX RÉGNER

- **Principe :**

- *Le tri rapide est fondé sur le principe de 'Diviser Pour Mieux Régner' :*
- **Diviser** : *Le tableau $tab[indinf..indsup]$ est partitionné 'Réarrangé' en deux sous tableaux non vides : $tab1[indinf..indpivot]$ et $tab2[indpivot+1..sup]$*
- *Tel que : chaque élément de $tab[indinf..indpivot]$ soit inférieur ou égal à chaque élément de $tab[indpivot+1..indsup]$. L'indice $indpivot$ est calculé pendant la phase de partitionnement.*



- $tab[indinf..indpivot-1] \leq tab[indpivot]=pivot < tab[indpivot+1..indsup]$

APPROCHE DIVISER POUR MIEUX RÉGNER

- **Régner** : *Les deux sous tableaux $tab1[indinf..indpivot-1]$ et $tab2[indpivot+1..sup]$ sont triés par des appels récursifs à la procédure principale du tri rapide.*
- **Combiner** : *Comme les deux sous tableaux sont triés sur place, aucun travail n'est nécessaire pour les re-combiner.*
- *Le tableau $tab[indinf..indsup]$ est trié.*
- **Remarque**
- *L'élément $pivot=tab[indpivot]$ est correctement rangé.*

ALGORITHME DE TRI RAPIDE

Procédure triRapide(indinf, indsup, tab)

P.F : indinf, indsup : Entier (E)

tab : tableau (E/S)

Début

Si indinf < indsup **Alors**

 indpivot ← partitionner(indinf, indsup, tab)

 triRapide(indinf, indpivot-1, tab)

 triRapide(indpivot+1, indsup, tab)

Finsi

Fin

PARTIONNER UN TABLEAU

Principe

- On choisit un élément appelé pivot qui appartient au tableau $tab[indinf..indsup]$
- En suite, on effectue des permutations afin de classer les éléments du tableau par rapport à ce pivot.
- On souhaite obtenir :
- $tab[indinf..indpivot-1] \leq pivot = tab[indpivot] < tab[indpivot+1..indsup]$

PARTIONNER UN TABLEAU

Choix du pivot

- Si on ne connaît aucune propriété particulière sur les éléments du tableau, le choix du pivot est arbitraire.

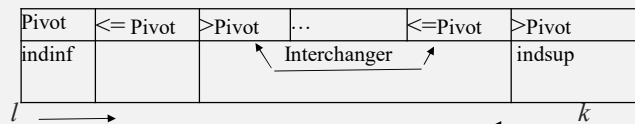
Exemple

- $Pivot = tab[indinf]$
- $= tab[indsup]$
- $= tab[(indinf+indsup)/2]$
- $= etc$

PARTIONNER UN TABLEAU

Action

- On place les éléments plus petits que le pivot dans la région inférieure du tableau et les éléments les plus grands dans la région supérieure.



l s'arrête des que $tab[l] > pivot$

k s'arrête des que $tab[k] \leq pivot$

Et on interchange $tab[l]$ et $tab[k]$

Au moment ou l et k se croisent c'est à dire $l > k$ alors, on a terminé : c'est à dire on a trouvé le pivot (élément trié dans le tableau).

EXEMPLE DE PARTITIONNEMENT

- Tab =

8	15	4	9	16	12	5	0
---	----	---	---	----	----	---	---


- Tab =

8	0	4	9	16	12	5	15
---	---	---	---	----	----	---	----


- Tab =

8	0	4	5	16	12	9	15
---	---	---	---	----	----	---	----


- Tab =

5	0	4	8	16	12	9	15
---	---	---	---	----	----	---	----

ALGORITHME PARTITIONNER : PREMIÈRE VERSION

Partitionner(indinf, indsup, tab) → Entier

Paramètres Formels

Indinf, indsup : Entier(E)

Tab : tableau (E/S)

Variables Intermédiaires :

l, k, indpivot : entier

pivot : TElement

Début

```

indpivot ← indinf
pivot ← tab[indpivot]
l ← indinf + 1
k ← indsup

```

ALGORITHME PARTITIONNER : PREMIÈRE VERSION

```

Tantque (l<=k) Faire
  Si (tab[l] <= pivot) Alors
    l ← l + 1
  Sinon
    Si (tab[k]>pivot) Alors
      k ← k - 1
    Sinon
      Si (l<k) Alors
        Permutation(l, k, tab)
        l ← l + 1
        k ← k - 1
      Finsi
    Finsi
  Finsi
Fintantque
Si indpivot ≠ k alors
  Permutation(indpivot, k, tab)
finsi
renvoyer k
Fin

```

ALGORITHME PARTITIONNER : DEUXIÈME VERSION

```

Partitionner(indinf, indsup, tab) → Entier
Paramètres Formels
  indinf, indsup :Entier(E)
  tab : tableau (E/S)
Variables Intermédiaires :
  l, k , indpivot : Entier
  pivot : TElement
Début
  indpivot ← indinf
  pivot ← tab[indpivot]
  l ← indinf + 1
  k ← indsup

```

ALGORITHME PARTITIONNER : DEUXIÈME VERSION

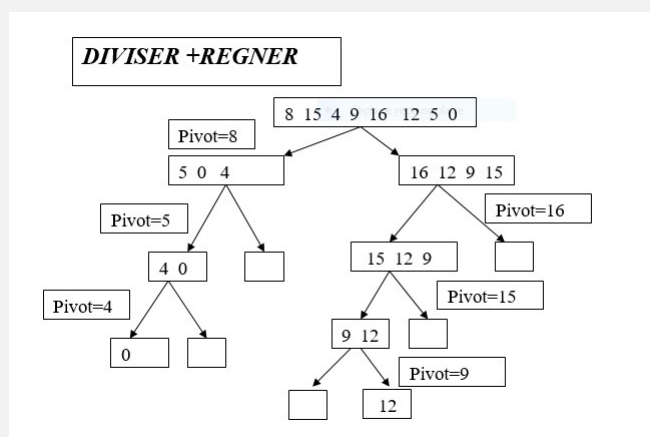
```

tantque ( $l \leq k$ ) Faire
  tantque ( $l \leq k$ ) et ( $\text{tab}[l] \leq \text{pivot}$ ) faire
     $l \leftarrow l + 1$ 
  fintantque
  tantque ( $l \leq k$ ) et ( $\text{tab}[k] > \text{pivot}$ ) faire
     $k \leftarrow k - 1$ 
  fintantque
  Si ( $l < k$ ) Alors
    permutation( $l, k, \text{tab}$ )
     $l \leftarrow l + 1$ 
     $k \leftarrow k - 1$ 
  Finsi
Fintantque
Si  $\text{indpivot} \neq k$  alors
  Permutation( $\text{indpivot}, k, \text{tab}$ )
finsi
renvoyer  $k$ 
Fin

```

13

EXEMPLE : TRI RAPIDE



14